Software Architecture and Design Specification

for

Student Attendance Management System (SAMS)

Version: 1.1

Status: Draft

Date: 29-10-2026

Authors: Team 1

Name	**SRN**
Cheethirala Sai Santosh	PES1UG24CS127
Deepak B	PES1UG24CS143
Epari Subhransi	PES1UG24CS161
Gyaan S Shetty	PES1UG24CS178

Revision History

Version	Date	Author	Change Summary
1.0	05-10-2026	Team 1	Initial version
1.1	29-10-2026	Team 1	Data model and API updated to match implementation

Approvals

Role	Name	Signature/Date
Team Lead	Gyaan S Shetty	
Developer	Cheethirala Sai Santosh	
Developer	Deepak B	
Tester	Epari Subhransi	
Faculty Guide		

1. Introduction

1.1 Purpose

This document describes the architecture and design of the Student Attendance Management System (SAMS) version 1.0. It explains how the requirements in the SRS will be implemented.

1.2 Scope

Covers all modules of SAMS: user login and management, subject and class management, attendance marking, reports, shortage email and leave requests.

1.3 Audience

Developers and testers in Team 1, the faculty evaluator, and anyone who maintains the system later.

1.4 Definitions

Term	Meaning

SAMS	Student Attendance Management System
MVT	Model View Template (Django pattern)
ORM	Object Relational Mapping
CSRF	Cross Site Request Forgery
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
SMTP	Simple Mail Transfer Protocol

2. Document Overview

2.1 How to use this document

Section 3 gives the architecture - components, pattern, technology, risks, traceability and security. Section 4 gives the design - sequence diagrams, API, error handling and UI. Requirement IDs (REQ-x, NFR-x) refer to the SRS.

2.2 Related Documents

- Software Requirements Specification (SRS) v1.0
- Software Test Plan (STP) v1.0
- Requirement Traceability Matrix (SRS Appendix C)

3. Architecture

3.1 Goals & Constraints

Goals:

- Pages load within 3 seconds for upto 100 users (NFR-1)
- Attendance for 70 students saved in under 2 seconds (NFR-2)
- Only authorised users can access data (SO-1)
- Easy to maintain - each module is a seperate Django app

Constraints:

- Python and Django must be used
- MySQL database

- Web application only
- Team of 4, one semester

3.2 Stakeholders & Concerns

Stakeholder	**Concerns**
Students	Correct attendance, privacy of thier data
Faculty	Fast and easy attendance marking, reports
Admin	Correct data, control over users and subjects
Developers	Modularity, easy to work in parallel
Evaluator	Traceability, security, documentation

3.3 Component (UML) Diagram

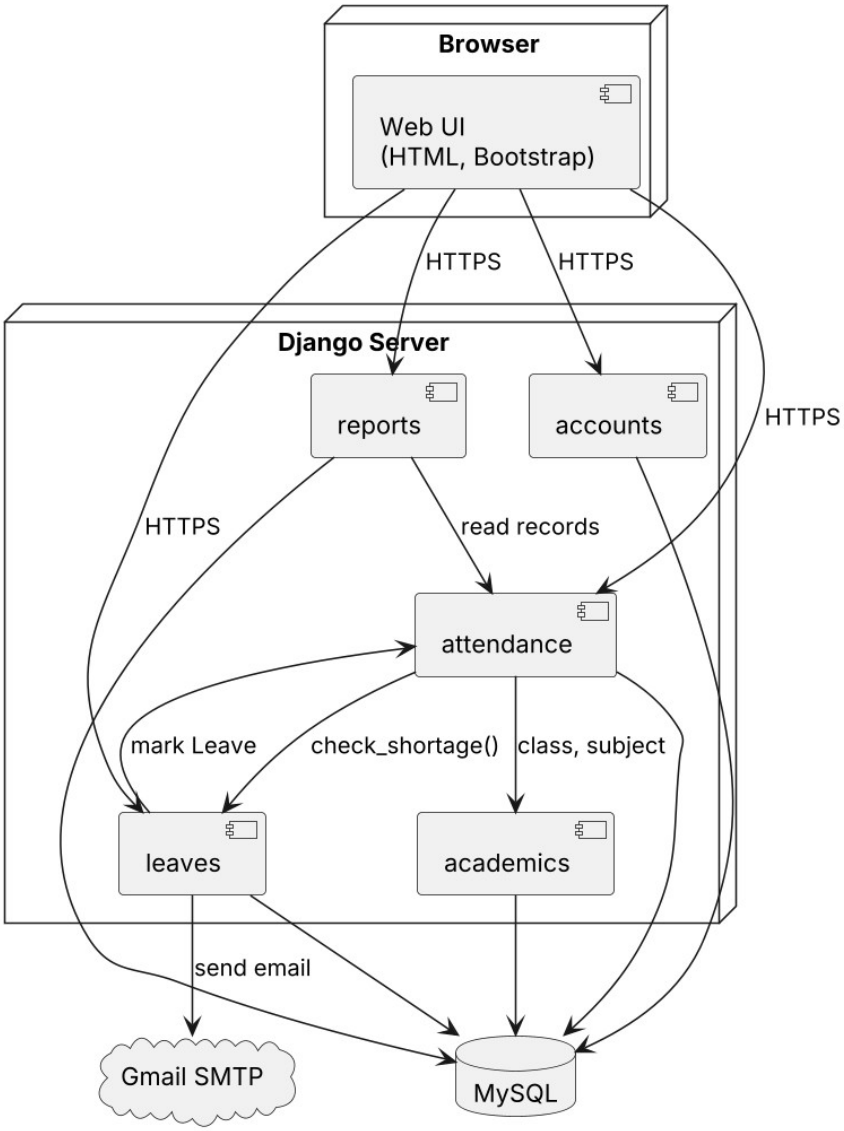


Figure 1: Component diagram of SAMS

3.4 Component Descriptions

Component	Responsibility	Owner
Web UI	HTML pages with Bootstrap, shown in the browser	All
accounts	Login, logout, roles, add/edit/delete users, Excel upload	Santosh
academics	Subjects, classes, enrolment, faculty assignment	Deepak
attendance	Mark and update attendance, calculate percentage, change log	Deepak

reports	Student attendance view, PDF/Excel reports, shortage list	Subhransi
leaves	Leave requests, approval, shortage email	Gyaan
MySQL	Stores all data	-
Gmail SMTP	External server used to send emails	-

3.5 Chosen Architecture Pattern and Rationale

We are using a Layered architecture with Django's MVT (Model-View-Template) pattern.

Layer	**Django part**	**Job**
Presentation	Templates	Show pages to the user
Business logic	Views	Handle requests, apply rules
Data	Models + ORM	Read/write MySQL

Why:

- Django already follows this pattern so it is easy to implement
- Each module is a seperate Django app, so each team member can work on thier own app
- Easy to test each layer

Rejected:

- Microservices - too complex for a small college system and a 4 person team
- React SPA + REST API - more front end work, server side templates are enough

3.6 Technology Stack & Data Stores

Layer	**Technology**
Front end	HTML, CSS, Bootstrap 5
Back end	Python 3.10, Django 4.2
Database	MySQL 8.0 (SQLite for local testing and CI)
Reports	reportlab (PDF), openpyxl (Excel)
Email	Gmail SMTP

Tools	GitHub, JIRA, VS Code

Main tables:

Table	Main Fields
User	id, username, password (hashed), name, email, role
Subject	code, name, semester
ClassSection	department, semester, section, students
Teaching	faculty, subject, class_section (which faculty teaches what)
AttendanceRecord	student, subject, class, date, hour, status (P/A/L), marked_by
AttendanceLog	record, changed_by, old_status, new_status, time
LeaveRequest	student, from_date, to_date, reason, status
ShortageAlert	student, subject, percent, sent_at (shortage mails sent)

3.7 Risks & Mitigations

Risk	Mitigation
Data loss if server crashes	Daily database backup (NFR-4)
Faculty marks attendance twice	Unique check on subject + class + date + hour (REQ-9)
Email not sent (Gmail limit / network)	Log failure and show on admin dashboard
Team members code conflicts	Each person works in own app and git branch, merge after review

3.8 Traceability to Requirements

Requirement	Component
REQ-1 to REQ-4	accounts
REQ-5 to REQ-7	academics

REQ-8 to REQ-11	attendance
REQ-12 to REQ-15	reports
REQ-16 to REQ-19	leaves
NFR-6, NFR-7, NFR-9	accounts
NFR-8	reports
NFR-10	All (Django CSRF + ORM)
NFR-11	attendance (AttendanceLog)

3.9 Security Architecture

Threat modeling done using STRIDE:

Threat	**Example in SAMS**	**Mitigation**
Spoofing	Someone logs in as another user	Hashed passwords, session timeout 30 min
Tampering	Student changes attendance	Only faculty of that subject can edit, role check on every view
Repudiation	Faculty says they did not change a record	AttendanceLog stores who changed what and when
Information disclosure	Student sees others attendance	Student can only query own records, HTTPS
Denial of service	Too many login attempts	Limit login attempts
Elevation of privilege	Student opens admin pages	@login_required and role decorators on all views

4. Design

4.1 Design Overview

Each Django app has the same structure:

File	**Contents**

models.py	Database tables
views.py	Request handling and business rules
urls.py	URL routes
forms.py	Input forms and validation
templates/	HTML pages
tests.py	Unit tests

4.2 UML Sequence Diagrams

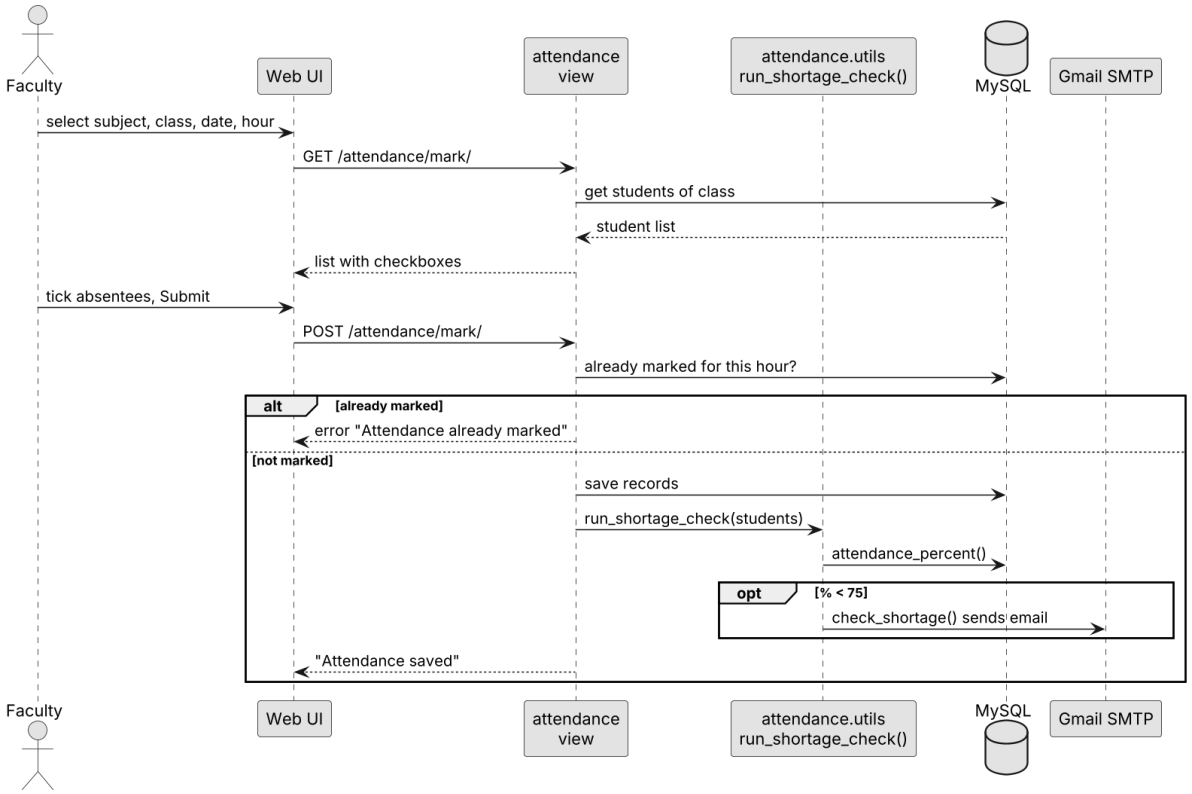


Figure 2: Faculty marks attendance (REQ-8, REQ-9, REQ-16)

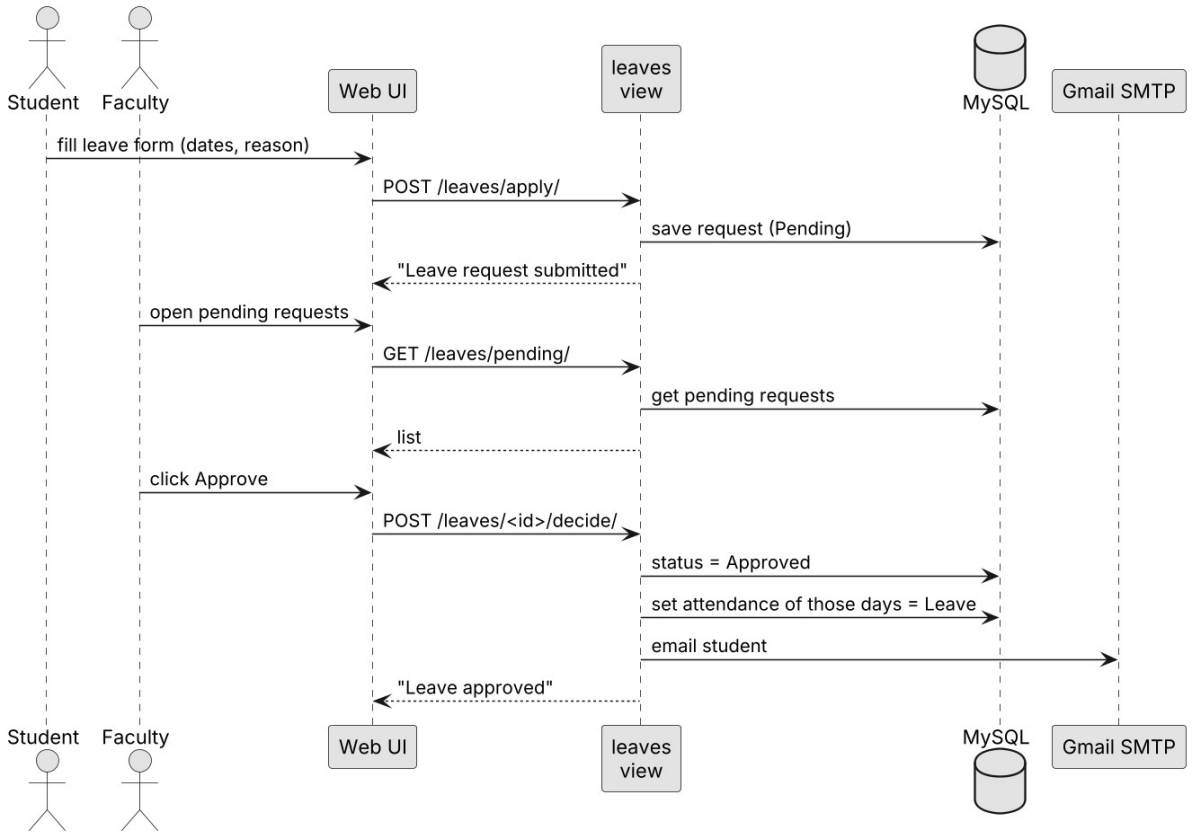


Figure 3: Student applies for leave and faculty approves (REQ-17 to REQ-19)

4.3 API Design

Interface definitions for the attendance and leaves components. All endpoints need login.

Attendance component

Endpoint	Type	Request	Response	Errors
/attendance/mark/	GET	teaching, date, hour	Student list page	not your class - not listed
/attendance/mark/	POST	teaching, date, hour, absent[]	Redirect + "Attendance saved"	"Attendance already marked"; future date
/attendance/edit/	POST	teaching, date, hour, status per student	"Attendance updated"	more than 3 days old (faculty)

Leaves component

Endpoint	Type	Request	Response	Errors
/leaves/apply/	POST	from_date, to_date, reason	"Leave request submitted"	invalid / overlapping dates
/leaves/pending/	GET	-	List of pending requests	403 not faculty
/leaves/my/	GET	-	Own leave requests	403 not student
/leaves/<id>/decide/	POST	decision (approve / reject)	"Leave approved / rejected"	403 not faculty; 404 not found

Reports component

Endpoint	Type	Request	Response	Errors
/reports/my/	GET	-	Subject wise % of logged in student	403 not student
/reports/class/	GET	teaching, start, end	Class report with %	invalid dates; 403 student
/reports/shortage/	GET	teaching (optional)	Students below 75%	403 student

Accounts component

Endpoint	Type	Request	Response	Errors
/accounts/ students/upload/	POST	.xlsx file	Rows added + rejected rows with reason	wrong file / header; 403 not admin

4.4 Error Handling, Logging & Monitoring

- Form errors shown next to the field (Django forms)

- Custom 403, 404 and 500 pages with simple messages
- Database saves done in a transaction so partial data is not saved
- Python logging used for errors - passwords and personal data are never logged
- Failed emails logged and shown to admin
- Admin dashboard shows count of failed logins and failed emails

4.5 UX Design

- Simple Bootstrap layout, works on mobile and laptop
- Dashboard according to role after login
- Attendance page - all students present by default, faculty ticks only absentees
- Attendance % shown with label "Shortage" if below 75%
- Clear success and error messages after every action

4.6 Open Issues & Next Steps

- QR code based attendance
- SMS alerts along with email
- Integration with university ERP

5. Appendices

5.1 Glossary

See Section 1.4 and SRS Appendix A.

5.2 References

- IEEE 42010 - Architecture Description
- IEEE 1016 - Software Design Descriptions
- OWASP Top 10
- Django Documentation

5.3 Tools

- PlantUML (diagrams)
- GitHub
- JIRA
- VS Code